

深度 Q 网络 (DQN)

先讲一个故事

2013 年, DeepMind 发布了一篇论文, 做了一件听起来疯狂的事: 用同一套代码、同一组超参数, 让 AI 玩 Atari 游戏。不告诉它游戏规则, 只让它看屏幕像素和分数——就像一个对说明书一无所知的小孩, 坐在电视机前乱按按键。

到 2015 年, 升级版打赢了 49 款游戏中的大多数, 在《打砖块》《乒乓》等游戏上达到并超越人类水平。这个 AI 叫 **DQN (Deep Q-Network)**。

在 DQN 之前, 深度神经网络和强化学习之间有一道公认的鸿沟: **训练不稳定**。用神经网络逼近 Q 值函数, 训练往往发散。DQN 用两个优雅的工程技巧彻底解决了这个问题。

本章的核心问题: Q-learning 已经够好了, 为什么还需要 DQN?

为什么 Q 表不够——维度灾难

回顾 Q-learning 的更新规则:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

这个算法需要一张 **Q 表**: 对每一个状态-动作对 (s, a) 存一个数值。在离散格子世界里, 这没问题。但在 CartPole 里, 状态是 4 个连续数字 (小车位置、速度、杆角度、角速度), Q 表有无穷多行。

即使强行离散化, 问题也没解决:

每维度格数	CartPole Q 表条目数 (4 维, 2 个动作)
$b = 10$	$10^4 \times 2 = 20,000$
$b = 20$	$20^4 \times 2 = 320,000$
$b = 100$	$100^4 \times 2 = 200,000,000$

Atari 游戏的状态是 $84 \times 84 \times 4$ 的灰度图像帧, Q 表的条目数约为 $256^{84 \times 84 \times 4}$ ——连写都写不下 (如图 1)。

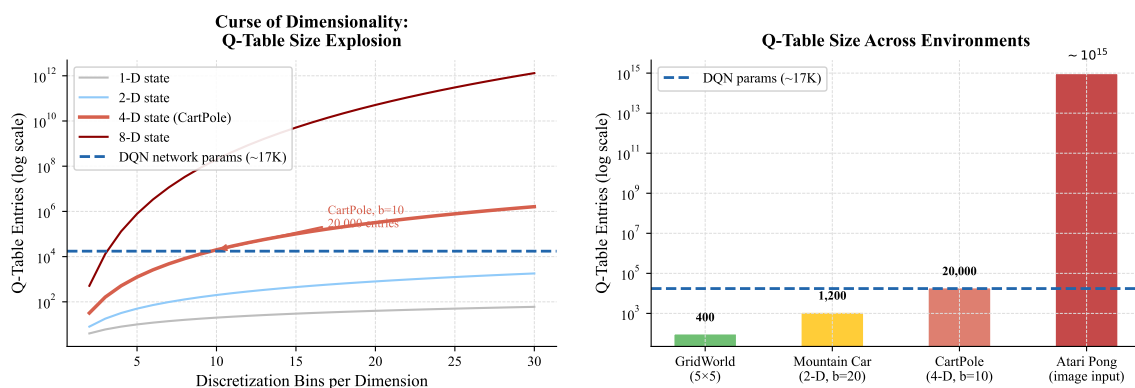


图 1: 维度灾难: Q 表大小随状态维度和离散化精度指数爆炸, 远超一个小型神经网络的参数量。

出路: 不要存一张表, 改用一个函数来拟合 $Q(s, a)$ 。

创新一：用神经网络逼近 Q 值

从 Q 表到 Q 网络

用参数为 θ 的神经网络代替 Q 表：

$$Q(s, a; \theta) \approx Q^*(s, a)$$

网络的输入是状态 s ，输出是**所有动作的 Q 值向量**——一次前向传播，同时得到所有动作的估值：

$$a^* = \arg \max_a Q(s, a; \theta)$$

损失函数

训练目标是让预测值逼近 Bellman 目标，损失为均方误差：

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

其中 $\mathcal{D}$ 是经验数据集， θ^- 是目标网络参数（后面详述）。

直接训练的两个致命问题

如果朴素地用每步转换 (s, a, r, s') 做梯度下降，会遇到：

- **问题一——时序相关性**：连续两帧的状态极度相似，梯度估计方差大，训练不稳定；
- **问题二——移动靶**：损失函数中的目标 $r + \gamma \max_{a'} Q(s', a'; \theta)$ 本身随 θ 变化——追一个会跑的靶， Q 值容易发散。

DQN 的后两个创新分别精确地解决这两个问题。

创新二：经验回放 (Experience Replay)

问题：时序相关

在线 Q-learning 每走一步就立刻更新网络。相邻两步的状态高度相关（小车向右倾 → 下一帧小车还在偏右），违反了随机梯度下降对样本独立同分布 (i.i.d.) 的假设，导致梯度方差大、训练震荡。

解法：回放缓冲区

把每步经验 $(s, a, r, s', \text{done})$ 存入先进先出的**回放缓冲区 $\mathcal{D}$** （容量 N ），训练时从中**随机均匀采样**一个小批量：

$$\{(s_i, a_i, r_i, s'_i)\}_{i=1}^B \sim \text{Uniform}(\mathcal{D})$$

为什么有效

- **打破时序相关**：随机采样使批次内样本近似 i.i.d.，满足梯度下降假设；
- **提高数据利用率**：同一条经验可被反复采样，相比在线学习不浪费；
- **稳定梯度**：批量大小 B 越大，梯度方差越小。

消融实验如图 2 所示：

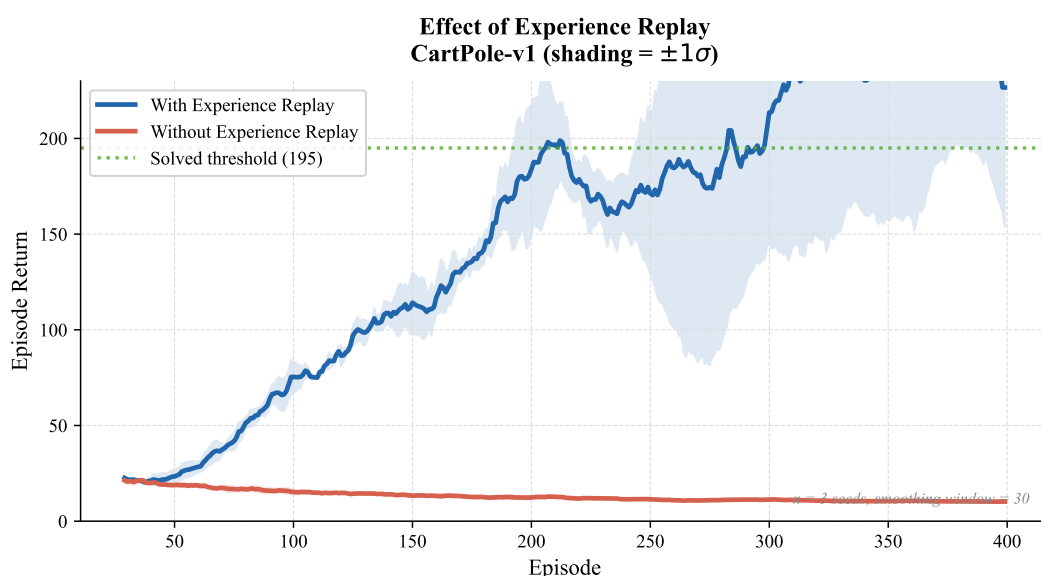


图 2: 经验回放的消融实验 (CartPole-v1, 3 个随机种子, 阴影为 $\pm 1\sigma$)。

创新三：目标网络 (Target Network)

问题：移动靶

朴素 DQN 的损失两边都含 θ ：

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta)}_{\text{目标}} - \underbrace{Q(s, a; \theta)}_{\text{预测}} \right)^2 \right]$$

每次梯度更新让预测值向目标靠近，但目标本身也随 θ 同步改变。这是**追移动靶**——更新一个 Q 值会间接拉动其他 Q 值，引发连锁不稳定，最终 Q 值发散。

解法：冻结目标网络

维护两个网络：

- **在线网络** $Q(s, a; \theta)$ ：每步梯度更新，用于选择动作；
- **目标网络** $Q(s, a; \theta^-)$ ：参数冻结，只参与 Bellman 目标的计算，每隔 C 步从在线网络硬同步一次：

$$\theta^- \leftarrow \theta \quad (\text{每隔 } C \text{ 步同步, 其余时刻冻结})$$

损失函数变为:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\underbrace{\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2}_{\text{固定目标 (短期内不动)}} \right]$$

直觉: 把「每步都在移动的靶」换成了「每 C 步挪一次的靶」——短期内靶静止, 训练稳定得多。 C 越大靶越稳, 但 θ^- 也越滞后, 通常取 $C \in [10, 1000]$ 。

消融实验如图 3 所示。

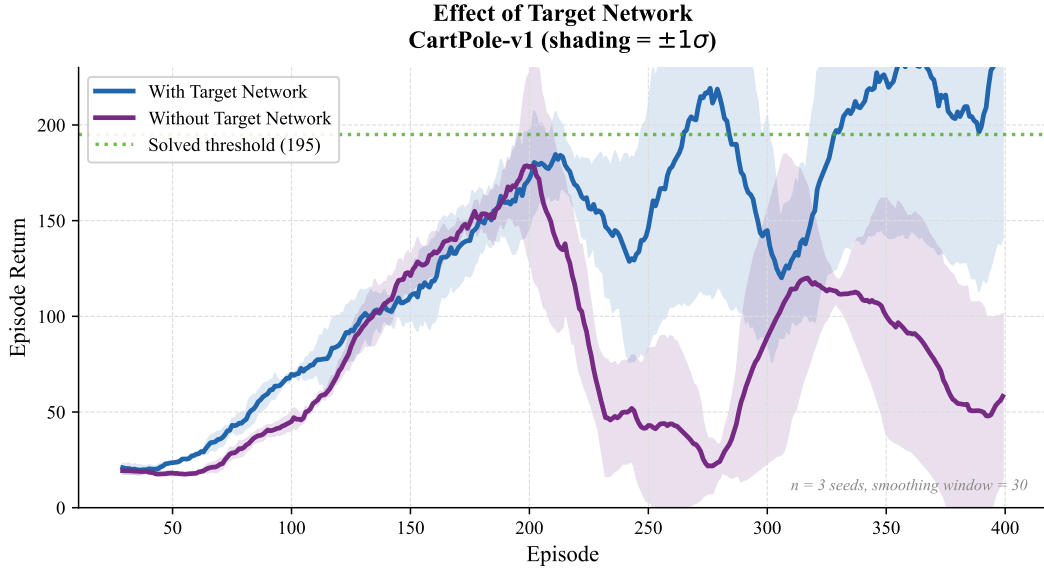


图 3: 目标网络的消融实验 (CartPole-v1, 3 个随机种子, 阴影为 $\pm 1\sigma$)。

完整 DQN 算法

综合三个创新:

1. **初始化:** 回放缓冲区 $\mathcal{D}$ (容量 N), 在线网络参数 θ , 目标网络 $\theta^- \leftarrow \theta$, 探索率 ϵ ;
2. **每个 episode**, 对每步 t :

(a) 以 ϵ -贪心选动作:

$$a_t = \begin{cases} \text{随机} & \text{以概率 } \epsilon \\ \arg \max_a Q(s_t, a; \theta) & \text{以概率 } 1 - \epsilon \end{cases}$$

(b) 执行 a_t , 观测 r_t, s_{t+1} ; 将 $(s_t, a_t, r_t, s_{t+1}, \text{done})$ 存入 $\mathcal{D}$;

(c) 从 $\mathcal{D}$ 随机采 B 条, 计算 Bellman 目标:

$$y_i = \begin{cases} r_i & \text{若终止} \\ r_i + \gamma \max_{a'} Q(s'_i, a'; \theta^-) & \text{否则} \end{cases}$$

(d) 梯度更新: $\theta \leftarrow \theta - \alpha \nabla_{\theta} \frac{1}{B} \sum_{i=1}^B (y_i - Q(s_i, a_i; \theta))^2$;

(e) 每 C 步执行: $\theta^- \leftarrow \theta$ 。

三个创新一句话总结:

问题	创新	核心操作
状态空间无限大	神经网络逼近	$Q(s, a; \theta) \approx Q^*(s, a)$
样本时序相关	经验回放	$(s, a, r, s') \sim \text{Uniform}(\mathcal{D})$
目标不稳定	目标网络	$\theta^- \leftarrow \theta$ 每 C 步

DQN 在 RL 体系中的位置

DQN 是深度强化学习的起点, 往前承接经典 RL, 往后开启整条深度 RL 主线:

1. **从表格到网络**: Q-learning (表格, 离散小状态) $\rightarrow$ DQN (神经网络, 连续高维状态), 更新规则完全相同, 只换了函数近似器;
2. **经验回放的深远影响**: 固定数据集 + 离线学习, 就是**离线强化学习 (Offline RL)**的核心范式——后续 CQL、IQL 等算法的数据均来自某种形式的回放缓冲区;
3. **目标网络的演化**: Hard update $\theta^- \leftarrow \theta \rightarrow$ Soft (Polyak) update $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$, 后者被 DDPG、TD3、SAC 广泛沿用;
4. **下一步**: Double DQN (解耦动作选择与评估, 缓解 Q 值高估)、Dueling DQN (将 Q 分解为状态价值 V 与优势函数 A) ——都是在 DQN 框架上的精准改进。

参考资料

1. Mnih, V., et al. (2013). Playing Atari with Deep Reinforcement Learning. *NeurIPS Deep Learning Workshop*.
2. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
3. Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapters 9–10.
4. 动手学强化学习 (Hands-on-RL) . 第 7 章: DQN 算法.